

Announcement

We will have Quiz #2 on Tuesday, April 14.

Open book and open notes

Final Project Presentation Schedule

April 16	April 21	April 23
James	Ty & JC (13+2)	Joseph (11+2)
Boshi	Thomas (11+2)	Sofia (11+2)
Supriya	Devon & Jean-Luc (13+2)	Cade (11+2)
Amirreza	Vigneswaran (11+2)	Bharat (11+2)
		Nitin (11+2)

Please submit your presentation slides to Blackboard before your presentation.

Course Evaluation

The course evaluation is available in Blackboard.

Please complete online course evaluation at Blackboard, which closes in April 27

Your feedback is crucial to improving the course

Today's Agenda

Texture

Texture

Core problems:

- Texture segmentation
- Texture based recognition
 - Objects, materials, textures
- Texture synthesis
 - Create synthetic images, fill in holes, image editing using computer graphics
- Shape from texture

Key issue: representing texture

Representing Textures

Textures are made up of

- stylized subelements
- spatially repeated in meaningful ways

Representation:

- find the subelements
- represent their statistics



Examples of texture images by a google search

Representing Textures - Subelements

What are the subelements, and how do we find them?

- Find subelements by applying filters, looking at the magnitude of the response
- Extreme case
 - template matching by normalized cross correlation

$$\frac{1}{MN} \sum_{x,y} \frac{(f(x,y) - \bar{f})(T(x,y) - \bar{T})}{\sigma_f \sigma_T}$$

Representing Textures

What filters?

- experience suggests **spots** and **oriented bars** at a variety of different scales
- Each filter corresponds to one pattern element

What statistics?

- The more the merrier
- At least, mean and standard deviation
- better, various conditional histograms.

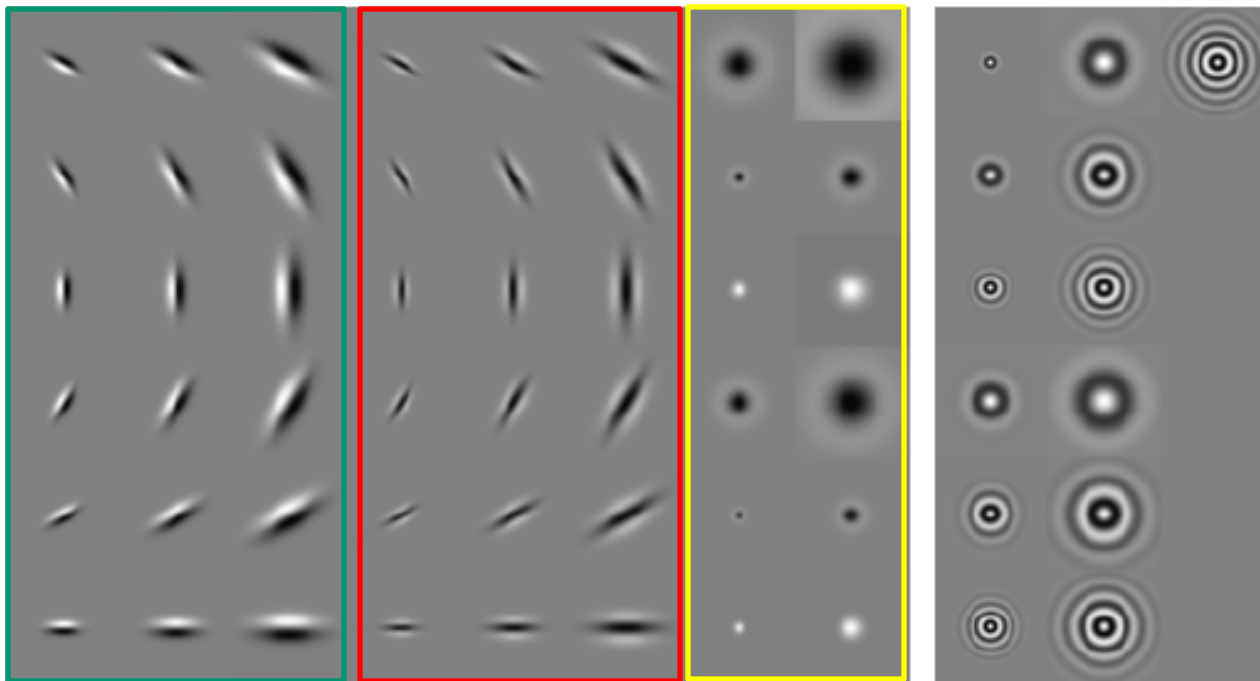
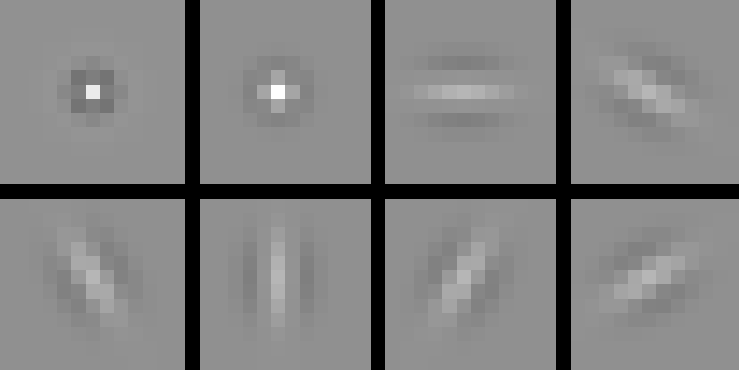
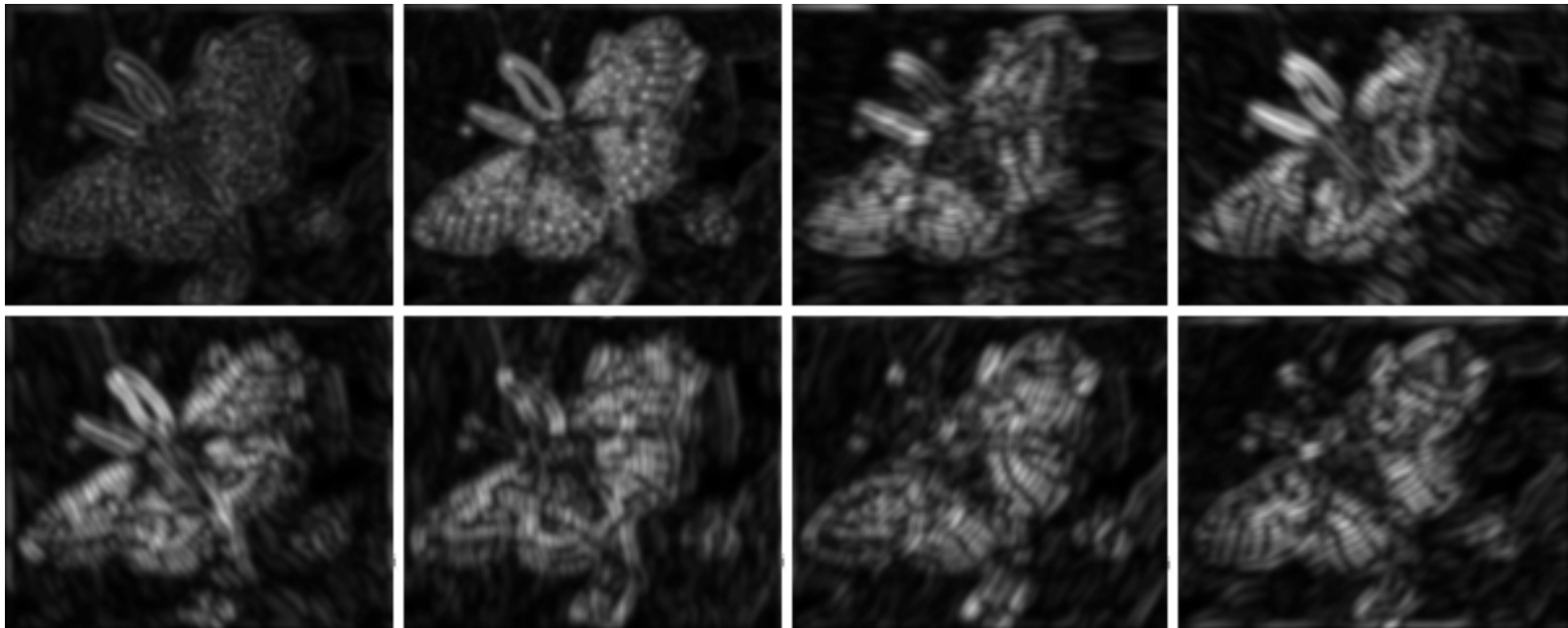
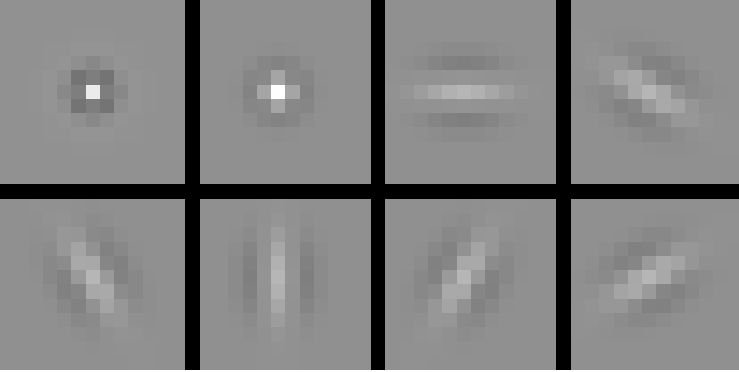


FIGURE 6.4: Left shows a set of 48 oriented filters used for expanding images into a series of responses for texture representation. Each filter is shown on its own scale, with zero represented by a mid-gray level, lighter values being positive, and darker values being negative. The left three columns represent edges at three scales and six orientations; the center three columns represent stripes; and the right two represent two classes of spots (with and without contrast at the boundary) at different scales. This is the set of filters used by Leung and Malik (2001). Right shows a set of orientation-independent filters, used by Schmid (2001), using the same representation (there are only 13 filters in this set, so there are five empty slots in the image). The orientation-independence property means that these filters look like complicated spots.

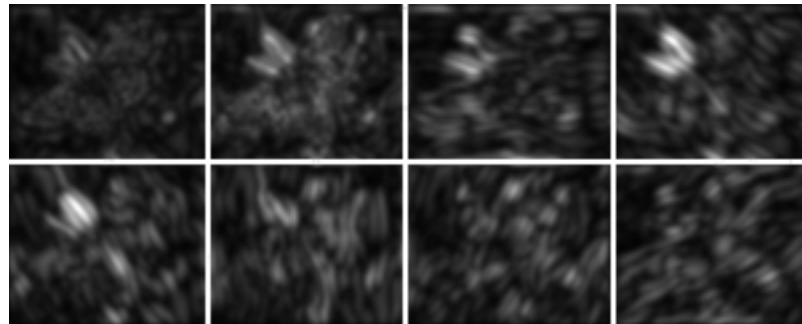


Squared response

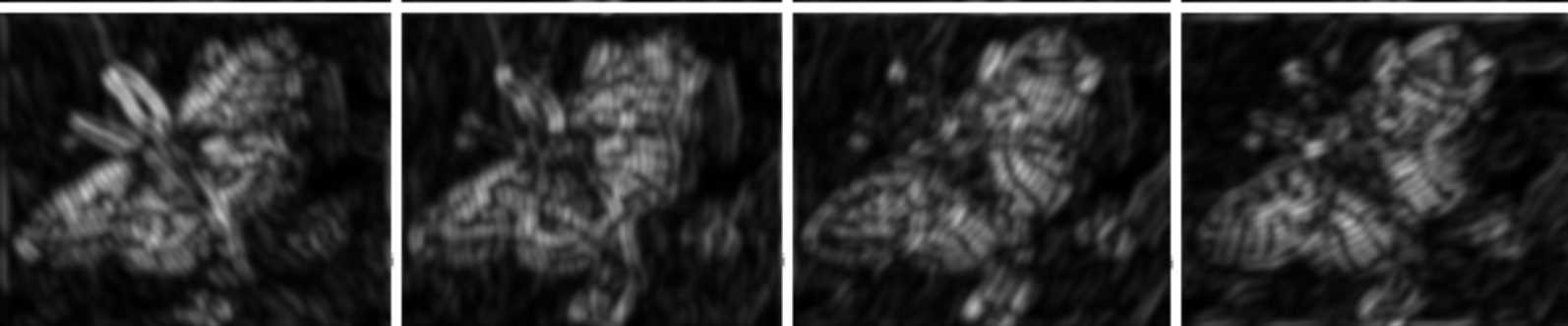
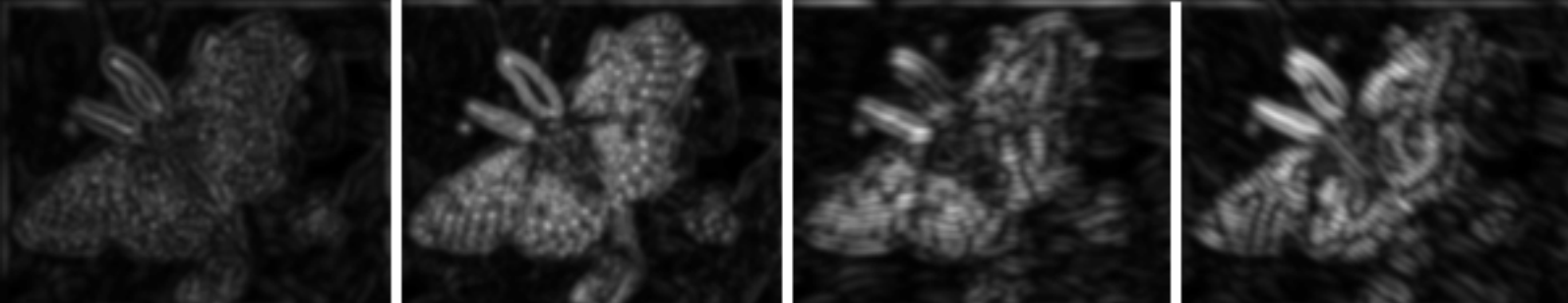




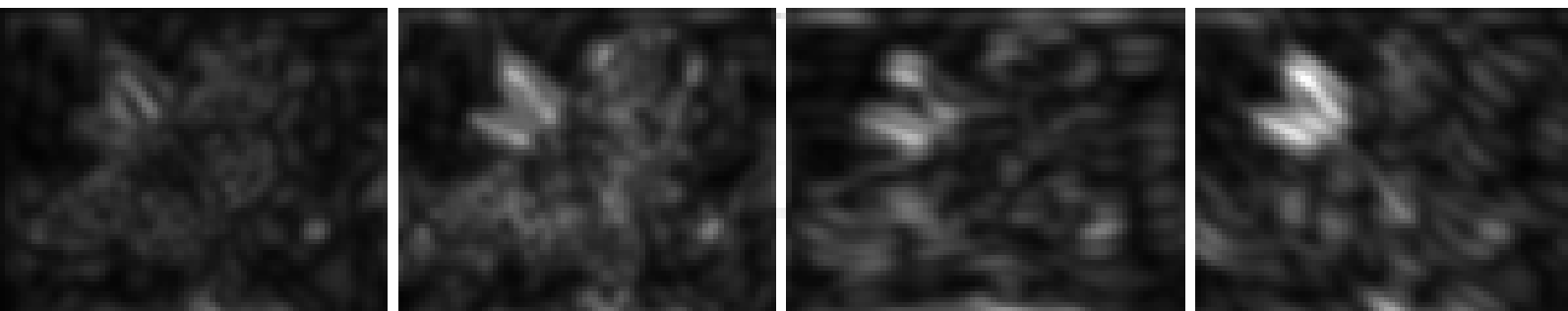
What happen if scale changes?



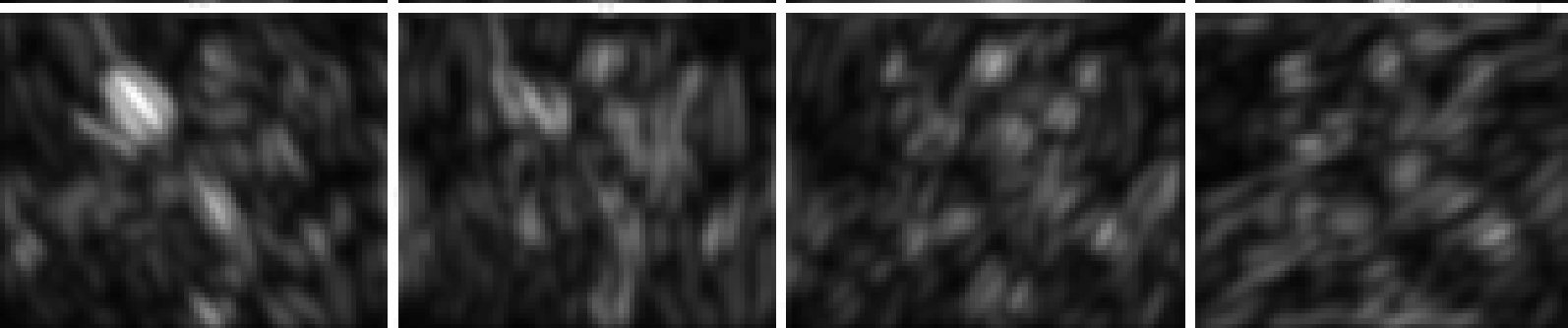
squared response



Fine



Coarse



Final Texture Representation

Form an oriented pyramid (or equivalent set of responses to filters at different scales and orientations).

Square the output

Take statistics of responses

- mean of each filter output
- Standard deviation of each filter output

Example based Texture Representations

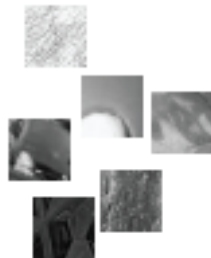
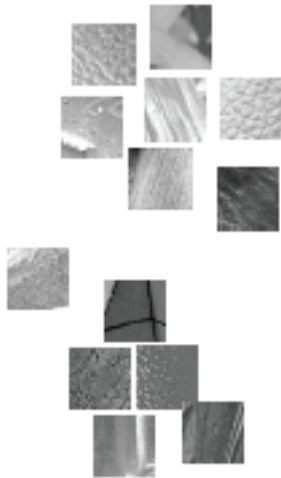
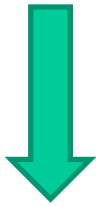
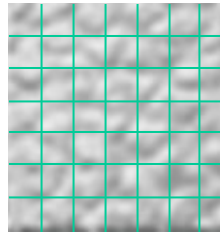
How does one choose the filters?

Solution

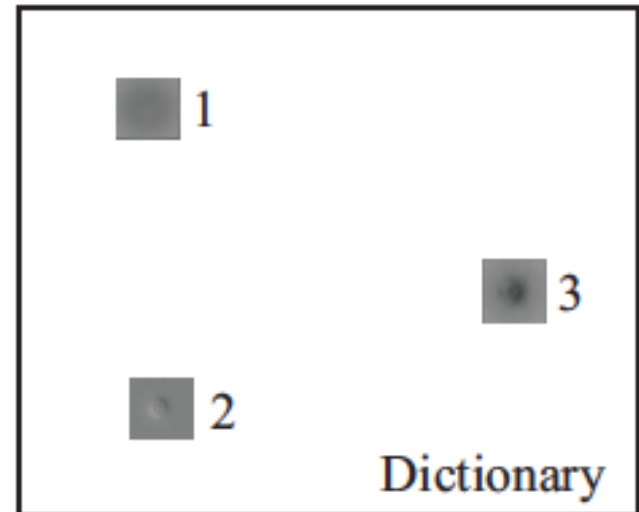
- build a dictionary of subelements from pictures
- describe the image using this dictionary

Building a Dictionary

Cut region into patches



Cluster
→



Learning a dictionary

Clustering the Examples

K-means

- represent patches with
 - intensity vector
 - vector of filter responses over patch

Choose k data points to act as cluster centers

Until the cluster centers change very little

Allocate each data point to cluster whose center is nearest.

Now ensure that every cluster has at least

one data point; one way to do this is by

supplying empty clusters with a point chosen at random from points far from their cluster center.

Replace the cluster centers with the mean of the elements in their clusters.

end

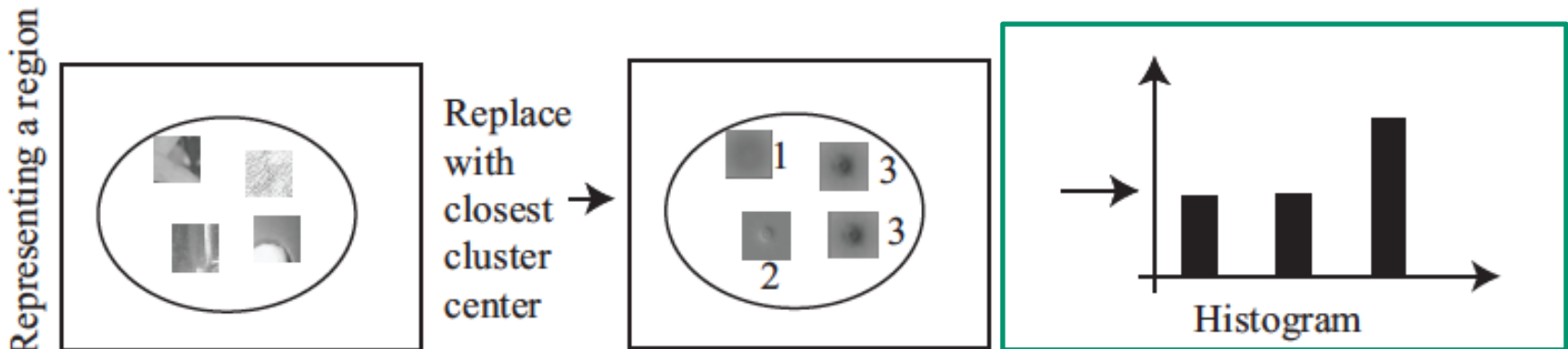
Representing a Region

Cut region into patches

Vector Quantization

- Represent a high-dimensional data item with a single number
 - Find and use the index of the nearest cluster center in dictionary
- Visual words – a vector of quantized image patches

Build histogram of resulting numbers



Representing a Region

Build a dictionary:

- Collect many training example textures

- Construct the vectors x for relevant pixels; these could be
 - a reshaping of a patch around the pixel, a vector of filter outputs computed at the pixel

- Obtain k cluster centers c for these examples

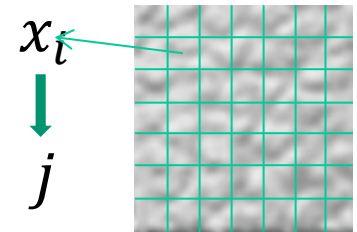
Represent an image domain:

- For each relevant pixel i in the image

 - Compute the vector representation x_i of that pixel

 - Obtain j , the index of the cluster center c_j closest to that pixel

 - Insert j into a histogram for that domain



Algorithm 6.2: Texture Representation Using Vector Quantization.

Texture Representations

Summarizing the trends in pattern elements

- either overall trend in filter responses
- or histogram of vector quantized patches

At a pixel

- compute representations for a patch centered on the pixel

For a region

- compute representations for the whole region

Texture Synthesis

Problem:

- Take a small example image of pure texture
- Use this to produce a large domain of “similar” texture

Why:

- Computer graphics demands lots of realistic texture
- Fill in holes in images created by removing objects

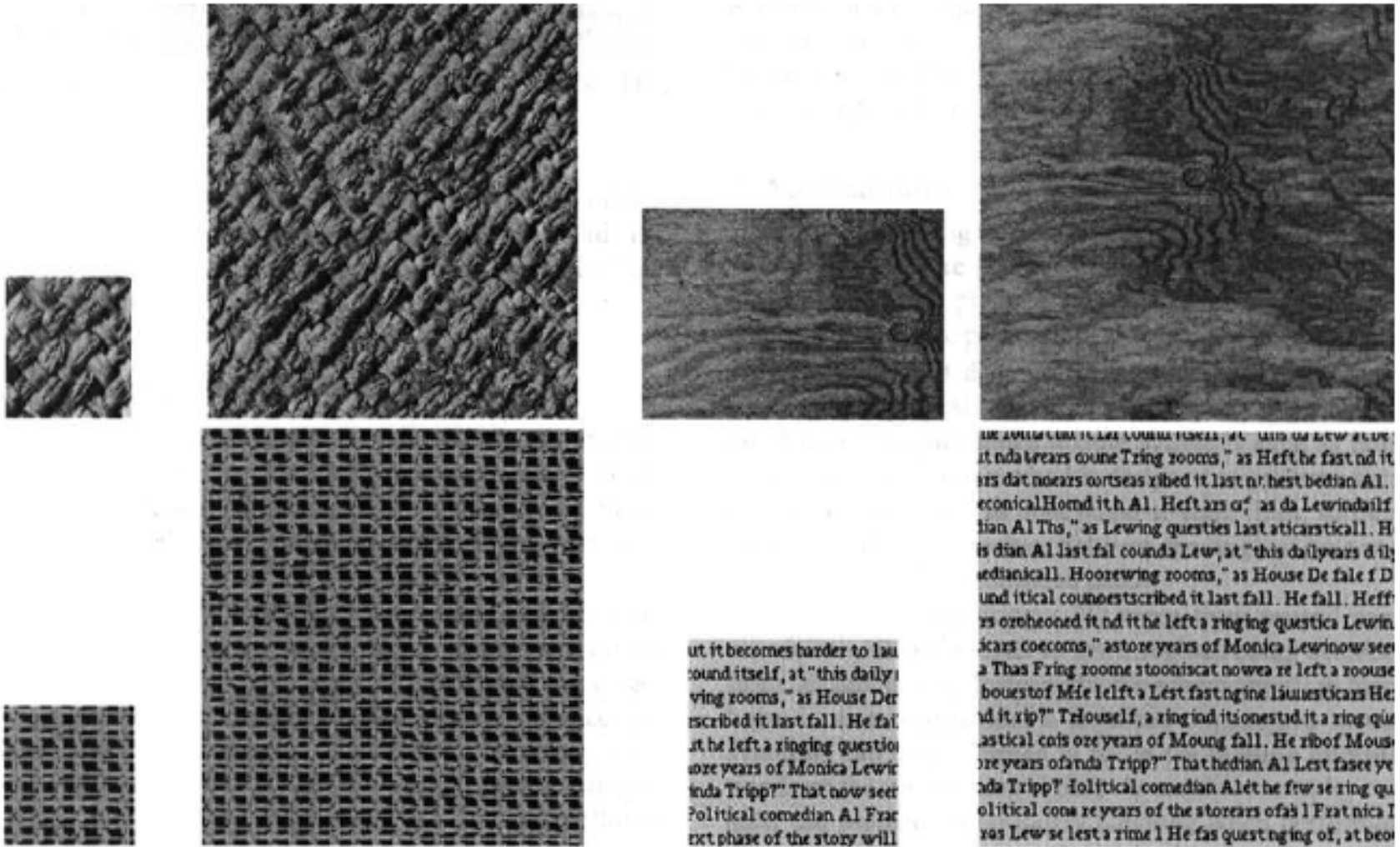
Texture synthesis

Simple case -- Fill holes

- Synthesize a single pixel in a large image
- Approach:
 - Match the window around that pixel to other windows in the image
 - Choose a value from the matching windows
 - most likely, uniformly and at random

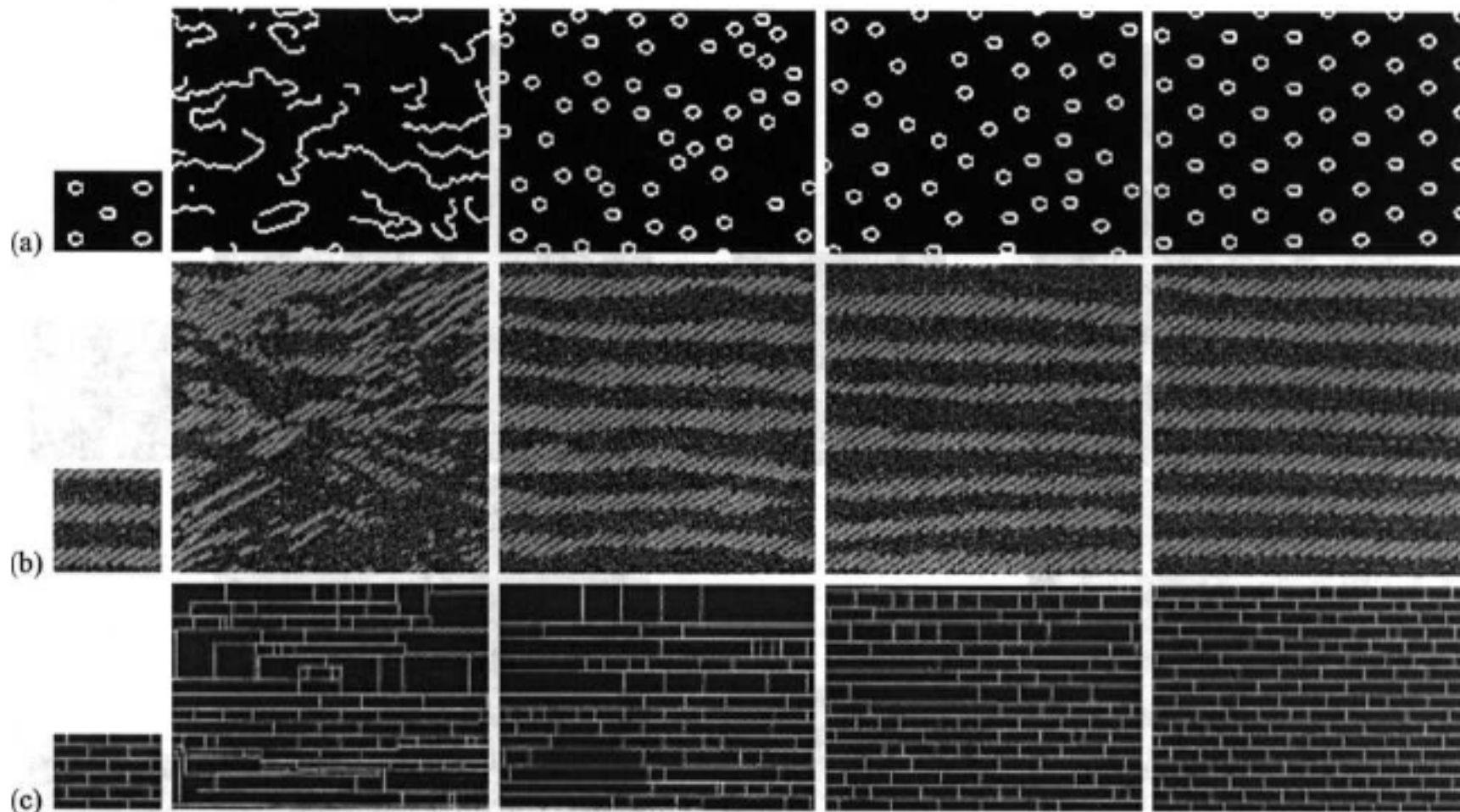
Expand to large images

- Start: take a piece of the example image
- Fill in pixels on the boundary
- Each time you fill in a pixel, you can use that to match



Small blocks are examples, large are synthesized. Notice how (for example) synthesized text looks like actual text.

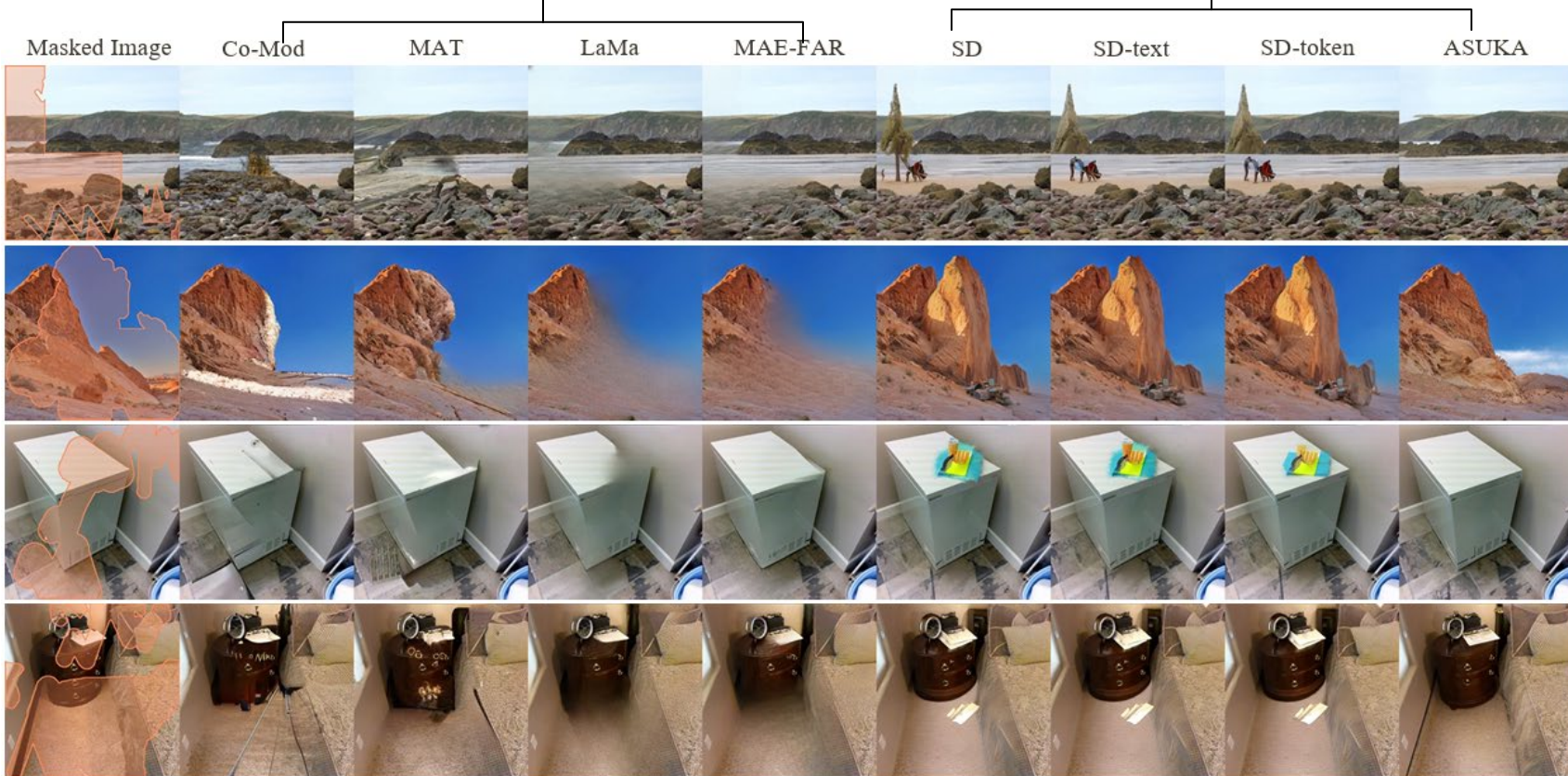
Neighborhood size



Recent Deep Learning based Image Inpainting Methods

GAN based

Stable Diffusion based



Wang et. Al, “Towards Enhanced Image Inpainting: Mitigating Unwanted Object Insertion and Preserving Color Consistency, “ CVPR 2025